

Alien Farm Defense — Rapport PCII

Alien Farm Defense

Rapport de projet de programmation concurrente et interface interactive

Université Paris-Saclay L3 Informatique — PCII 2026

Table des matières

1. [Introduction](#)
 2. [Analyse globale](#)
 3. [Plan de développement](#)
 4. [Conception générale](#)
 5. [Conception détaillée](#)
 6. [Résultats](#)
 7. [Documentation utilisateur](#)
 8. [Documentation développeur](#)
 9. [Conclusion et perspectives](#)
-

1. Introduction

Ce projet a été réalisé dans le cadre du module de **Programmation Concurrente et Interfaces Interactives** (L3 Informatique, Université Paris-Saclay). L'objectif est de développer un jeu vidéo original en **Java/Swing** intégrant les concepts de **programmation concurrente** et d'**interfaces interactives** vus en cours.

Le jeu, intitulé **Alien Farm Defense**, se déroule sur une carte vue du dessus en deux zones. Le joueur incarne un fermier qui gère son élevage de vaches pour accumuler de l'argent, s'approvisionne au marché en achetant des armes et du bétail, puis doit défendre sa ferme contre des vagues d'extraterrestres qui tentent d'enlever ses vaches. Une barre de progression temporelle rythme chaque niveau : des attaques intermédiaires ponctuent la montée en tension, avant un combat de boss final qui conditionne la victoire ou la défaite. Entre deux niveaux, une boutique d'améliorations permet d'investir ses gains

dans des upgrades permanentes.

Ce projet a permis de mettre en pratique l'architecture **MVC**, la **boucle de jeu à 60 FPS** via un `Timer Swing`, la gestion de l'état global d'une partie à travers une machine à états, ainsi que la conception d'un système de rendu avec caméra, tile-map et sprites animés.

2. Analyse globale

L'analyse du cahier des charges a fait émerger **9 fonctionnalités principales**, classées par priorité et niveau de difficulté.

F1 — Déplacement du joueur et caméra

Le joueur se déplace librement sur un monde de $2\,000 \times 1\,400$ pixels à l'aide des touches directionnelles. Une caméra suit le joueur et convertit les coordonnées du monde en coordonnées écran. Le monde est clampé aux bords pour que la caméra ne dépasse jamais les limites.

- **Difficulté** : Moyenne
- **Priorité** : Haute

F2 — Monde tilemap et zones

Le monde est divisé en deux zones de $1\,000$ px chacune : la ferme (gauche) et le marché (droite). Chaque zone est rendue par une grille de 25×35 tuiles (40 px/tuile) chargées depuis des fichiers texte. Un trait animé bleu marque la frontière.

- **Difficulté** : Moyenne
- **Priorité** : Haute

F3 — Ferme et cycle de vie des vaches

Les vaches achetées sont d'abord placées dans l'inventaire du joueur, puis déployées dans la ferme par clic gauche (quand le joueur est dans la zone ferme). Elles évoluent automatiquement selon trois états : **Bébé** → **Adulte** → **Productive**. En état productif, elles accumulent de la monnaie par cycle. La **récolte est automatique** : à chaque frame,

`Ferme.recolterTout()` est appelé et l'or est directement ajouté au joueur.

- **Difficulté** : Moyenne
- **Priorité** : Haute

F4 — Système économique (monnaie et marché)

Le joueur dispose d'une monnaie initiale, gagnée par la récolte et les récompenses de combat. Il la dépense au marché auprès de quatre vendeurs (forge, élevage, apothicaire, armurerie) accessibles par proximité. Chaque achat prend un temps incompressible modélisé par une `ActionDuree`.

- **Difficulté** : Moyenne
- **Priorité** : Haute

F5 — Système de combat : vagues aliens et boss

À des instants prédéfinis sur la barre de progression, des vagues d'aliens apparaissent depuis le bord droit de l'écran et marchent vers la ferme. Le combat est automatique (basé sur des cooldowns). En cas de défaite, l'alien repart avec une vache. À la fin du timer, un boss final détermine la victoire ou la défaite du niveau.

- **Difficulté** : Élevée
- **Priorité** : Haute

F6 — Progression temporelle et gestion des niveaux

Une barre de progression compte le temps du niveau. Les vagues et le boss sont des `EvenementTemporel` planifiés à des positions précises sur cette barre. La difficulté augmente à chaque niveau : durée, nombre de vagues, PV et dégâts des aliens.

- **Difficulté** : Moyenne
- **Priorité** : Haute

F7 — Boutique d'améliorations (entre les niveaux)

Après chaque victoire, le jeu entre dans un état `UPGRADE_SHOP` où le joueur peut acheter des améliorations permanentes (PV max, dégâts d'arme, vitesse de croissance des vaches, or de départ) avant de passer au niveau suivant.

- **Difficulté** : Facile
- **Priorité** : Moyenne

F8 — Tableau des scores

Le jeu enregistre les 10 meilleurs scores sur le disque. À la fin d'une partie, le joueur saisit ses initiales (3 caractères). Le tableau est consultable depuis le menu principal.

- **Difficulté** : Facile
- **Priorité** : Moyenne

F9 — Système de succès

Cinq succès débloquables récompensent des accomplissements spécifiques (premier alien tué, 500g gagnés, 50 aliens éliminés, niveau terminé à PV max, 3 vaches simultanées). Les succès débloqués sont affichés comme badges dans la barre latérale.

- **Difficulté** : Facile
- **Priorité** : Basse

Tableau récapitulatif

#	Fonctionnalité	Difficulté	Priorité
F1	Déplacement joueur et caméra	Moyenne	Haute
F2	Monde tilemap et zones	Moyenne	Haute
F3	Ferme et cycle de vie des vaches	Moyenne	Haute
F4	Système économique (marché)	Moyenne	Haute
F5	Combat : vagues aliens et boss	Élevée	Haute
F6	Progression temporelle et niveaux	Moyenne	Haute
F7	Boutique d'améliorations	Facile	Moyenne
F8	Tableau des scores	Facile	Moyenne
F9	Système de succès	Facile	Basse

3. Plan de développement

Le développement s'est organisé en **6 séances** selon une approche incrémentale, en partant du socle MVC jusqu'aux fonctionnalités de polish.

Séance 1 — Squelette MVC, joueur et monde

- **Analyse** (30 min) : Étude de l'architecture MVC avec Swing, choix du Timer à 60 FPS, conception du système de coordonnées monde/écran.
- **Conception** (30 min) : Définition des classes `Joueur`, `Carte`, `Partie`, `EtatJeu`. Machine à états simplifiée.
- **Développement** (~3h) : `Main.java` + `VuePrincipale` (JFrame + boucle Timer), `Joueur` (position, vitesse, déplacement delta-time), `Camera` (centrer + clamber + conversion coordonnées), `ControleurJoueur` (KeyListener 4 directions), `Constantes.java`.
- **Tests** (20 min) : Déplacement fluide, joueur ne sort pas du monde, caméra clampée.

Séance 2 — Tilemap, ferme et vaches

- **Analyse** (20 min) : Étude du format tile-map texte, choix du pack Kenney Tiny Town.
- **Conception** (20 min) : Algorithme de rendu de tuiles avec culling caméra. Modèle `Animal` → `Vache` → `EtatVache`. Algorithme de croissance avec report de surplus.
- **Développement** (~4h) : `TileManager` (chargement PNGs 16×16 → 40×40), `MondeRenderer` (grille + culling), `farm_map.txt` / `market_map.txt`, `Animal` (abstrait), `Vache` (BEBE→ADULTE→PRODUCTIVE, accumulation monnaie), `Ferme` (troupeau, tick, récolte).
- **Tests** (30 min) : 12 `VacheTest` + 14 `FermeTest` (transitions d'état, surplus, récolte, capacité).

Séance 3 — Économie, marché et actions avec durée

- **Analyse** (20 min) : Conception du marché à vendeurs-objets monde, système de proximité (rayon 90 px), `ActionDuree` pour cooldown achat/récolte.
- **Conception** (20 min) : `ArticleMarche` , `ResultatAchat` , flux d'achat dans `ControleurMarche` . Architecture 4 vendeurs positionnés dans la zone marché.
- **Développement** (~3h) : `Marche` + `ArticleMarche` , `VendeurMarche` (position monde, rayon, items), `VueMarchePopup` (anneau pulsé + popup achat), `ControleurMarche` , `ActionDuree` + `VueActionJoueur` .
- **Tests** (20 min) : 9 `MarcheTest` + 10 `ActionDureeTest` + 10 `ControleurMarcheTest` .

Séance 4 — Combat, aliens et boss

- **Analyse** (30 min) : Étude de la mécanique de combat automatique par cooldown, animation aliens (approach → combat → départ), cycle phases vague/boss.
- **Conception** (30 min) : `Attaque` (échange de coups, gestion séquentielle des aliens), `AlienVisuel` (machine à états visuelle), `PhaseAttaque` enum.
- **Développement** (~4h) : `Arme` , `Extraterrestre` , `BossFinal` , `Attaque` , `AlienVisuel` + `EtatVisuel` , `ControleurAttaque` (phases + enlèvement vache), `ControleurCombat` (boss + récompense), `VueAliens` (sprites PNG manned), `VueCombat` (overlay HP).
- **Tests** (30 min) : 11 `CombatUnitTest` + 9 `AttaqueTest` + 9 `ControleurAttaqueTest` + 6 `ControleurCombatTest` .

Séance 5 — Progression temporelle, HUD et polish visuel

- **Analyse** (20 min) : Conception de la barre de progression avec événements planifiés, sprites directionnels joueur, indicateurs de navigation.
- **Conception** (20 min) : `EvenementTemporel` , `BarreProgression` (chrono + détection événements), `Niveau` (factory paramétrique).
- **Développement** (~4h) : `Niveau` , `BarreProgression` , `EvenementTemporel` , `ControleurJeu.tickProgression()` , `VueBarreProgression` (marqueurs triangulaires), `VueHUD` , `VueIndicateurs` (flèches de bord), sprites joueur (Blue Boy, 4 directions × 2 frames), sprites vaches (Bébé/Adulte/Productive), `CacheSpritesAliens` (UFOs manned PNG).
- **Tests** (20 min) : 11 `NiveauTest` + 12 `BarreProgressionTest` .

Séance 6 — Boutique, leaderboard, succès et états globaux

- **Analyse** (20 min) : Conception des 3 features de polish. `EtatJeu.UPGRADE_SHOP` , persistance scores sur disque, enum `Succes` avec seuils.
- **Conception** (20 min) : `Upgrades` (multiplicateurs), `TableauScores` (top 10, sérialisation), `GestionnaireSucces` (set + compteurs).
- **Développement** (~3h) : `Upgrades` , `VueUpgrades` (4 cartes, navigation ←/→, ENTRÉE/ESPACE), `TableauScores` + `VueTableauScores` , `Succes` (enum), `GestionnaireSucces` , hooks dans `ControleurJeu` / `ControleurAttaque` / `Joueur` .
- **Tests** (15 min) : Tests `PartieTest` (23 tests, transitions `UPGRADE_SHOP`).

Autres tâches (transversales)

- **Rédaction du rapport** : tout au long du projet (~3h).
- **Création / adaptation des assets** : sprites vaches custom, copie des assets aliens Kenney, tile-maps farm/market (~2h, réparties sur séances 2 et 5).
- **Tests de régression** : séance 6 (vérification globale, 179 tests au total).

Total estimé : ~26h.

Diagramme de Gantt

Le diagramme de Gantt détaillé est fourni en fichier séparé (docs/Gantt_AlienFarmDefense.xlsx). Le tableau ci-dessous en présente une vue synthétique.

Tâche	S1	S2	S3	S4	S5	S6
Squelette MVC + joueur + caméra	■					
Tilemap + ferme + vaches		■				
Économie + marché + actions			■			
Combat + aliens + boss				■		
Progression + HUD + polish visuel					■	
Boutique + leaderboard + succès						■
Tests (179 au total)	▤	▤	▤	▤	▤	■
Rapport	▤	▤	▤	▤	▤	■

4. Conception générale

Le projet suit le patron **Modèle-Vue-Contrôleur (MVC)** pour séparer rigoureusement les données, leur représentation et la logique d'interaction.

Structure des paquets

- **Modèle** (com.fermedefense.modele) : logique métier pure — état du joueur, cycle de vie des vaches, mécanique de combat, progression du niveau, achievements, scores.
- **Vue** (com.fermedefense.vue) : rendu graphique Swing — tile-map, sprites, overlays, HUD, indicateurs.
- **Contrôleur** (com.fermedefense.controleur) : boucle de jeu principale, gestion des événements clavier, orchestration des sous-systèmes.
- **Utilitaire** (com.fermedefense.utilitaire) : constantes globales.

Blocs fonctionnels et circulation de l'information

Le schéma ci-dessous présente l'architecture fonctionnelle du projet.

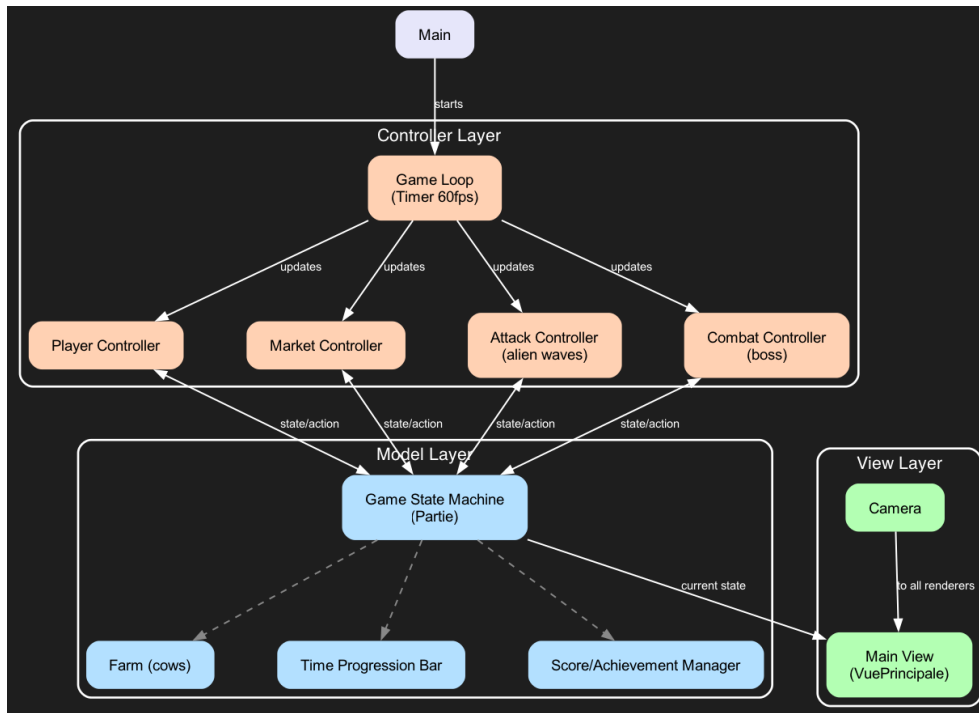


Diagramme de blocs fonctionnels

Description des blocs

1. **Main** : Point d'entrée. Instancie tous les objets modèle, crée la `vuePrincipale` et démarre la boucle.
2. **Boucle de jeu (Timer 60 FPS)** : Cœur de l'application. Appelle `tick(deltaMs)` à chaque frame. Orchestre tous les contrôleurs.
3. **Contrôleur joueur** : Lit les touches enfoncées et déplace le joueur dans le monde. Vérifie la zone (ferme/marché) pour autoriser les actions.
4. **Contrôleur marché** : Gère les achats (proximité vendeur, fonds suffisants, type d'article). Retourne un `ResultatAchat`.
5. **Ferme** : Met à jour toutes les vaches à chaque tick (croissance, production). Gère l'ajout, le retrait et l'enlèvement de vaches.
6. **Contrôleur attaque** : Déclenche les vagues d'aliens au signal de la barre de progression. Gère le cycle APPROCHE → COMBAT → DEPART.
7. **Contrôleur combat** : Gère le boss final avec le même cycle visuel. Accorde une récompense en monnaie si victoire.
8. **Progression** : Compte le temps du niveau, planifie les événements (vagues, boss), détecte leur déclenchement à chaque tick.
9. **Machine à états (Partie)** : Centralise l'état global du jeu (MENU, EN_COURS, COMBAT_FINAL,

UPGRADE_SHOP, VICTOIRE, DEFAITE).

10. **Vue principale** : `paintComponent` dessine le monde (tilemap + entités) puis les overlays appropriés selon l'état courant.
11. **Caméra** : Convertit les coordonnées monde en coordonnées écran. Centrée sur le joueur, clampée aux bords du monde.
12. **Gestionnaire de scores / succès** : Vérifie les seuils de déclenchement à chaque événement. Sauvegarde le tableau des scores sur disque.

Boucle de jeu et machine à états

```

Main.java
├── Crée modèle (Joueur, Ferme, Carte, Marche, Partie, Upgrades, ...)
├── VuePrincipale.lancer()
│   ├── Timer Swing (16ms ≈ 60 FPS)
│   │   └── tick(deltaMs)
│   │       ├── controleurJoueur.tick()           ← déplacement
│   │       ├── ferme.mettreAJour(deltaMs)        ← croissance vaches
│   │       ├── actionDuree?.tick(deltaMs)        ← cooldown achat/récolte
│   │       └── tickProgression(deltaMs)
│   │           ├── barreProgression.mettreAJour()
│   │           ├── → ATTAQUE_INTERMEDIAIRE → controleurAttaque
│   │           ├── → COMBAT_FINAL              → controleurCombat
│   │           └── fin barre                    → VICTOIRE / DEFAITE

```

États de la partie

```

MENU —demarrer()→ EN_COURS → COMBAT_FINAL → VICTOIRE → UPGRADE_SHOP → EN_COURS (niv+1)
                        |
                        |
                        └── terminer(false) → DEFAITE
COMBAT_FINAL
├── terminer(false) → DEFAITE

```

5. Conception détaillée

5.1 F1 — Déplacement du joueur et caméra

Cette fonctionnalité permet au joueur de se déplacer librement dans un monde de $2\,000 \times 1\,400$ pixels à l'aide des touches directionnelles, avec support du mouvement diagonal. Elle est répartie entre `Joueur` (modèle — position, vitesse,

directions actives), `ControleurJoueur` (contrôleur — `KeyListener` qui met à jour le `Set<Action>`), `Camera` (vue — conversion coordonnées monde ↔ écran) et `carte` (modèle — clamping des bords et détection des zones).

Structures de données

Joueur (modele/joueur/) — classe centrale du modèle joueur :

Attribut	Type	Description
<code>x</code> , <code>y</code>	<code>double</code>	Position dans le monde (pixels)
<code>vitesse</code>	<code>double</code>	Vitesse en pixels/seconde
<code>directionsActives</code>	<code>Set<Action></code>	Ensemble des directions simultanément enfoncées
<code>pointsDeVie</code>	<code>int</code>	PV actuels
<code>pointsDeVieMax</code>	<code>int</code>	PV maximum
<code>monnaie</code>	<code>int</code>	Or disponible
<code>totalMonnaieGagnee</code>	<code>int</code>	Cumulatif pour le succès <code>FERMIER_PROSPERE</code>
<code>inventaire</code>	<code>Inventaire</code>	Grille 5×5 d' <code>ObjetInventaire</code>
<code>indexArmeEquipee</code>	<code>int</code>	Index de l'arme active dans l'inventaire

Camera (vue/) :

Attribut	Type	Description
<code>offsetX</code> , <code>offsetY</code>	<code>double</code>	Décalage monde→écran

Carte (modele/jeu/) : dimensions du monde, zones, clamping de la position du joueur.

Constantes du modèle

Constante	Valeur	Description
<code>LARGEUR_CARTE</code>	2 000	Largeur totale du monde (px)
<code>HAUTEUR_CARTE</code>	1 400	Hauteur totale du monde (px)
<code>LARGEUR_VIEWPORT</code>	1 200	Largeur de la fenêtre de jeu
<code>HAUTEUR_VIEWPORT</code>	840	Hauteur de la fenêtre de jeu
<code>LARGEUR_SIDEBAR</code>	240	Largeur de la barre latérale
<code>PLAYER_SIZE</code>	48	Taille du sprite joueur (px)
<code>VITESSE_JOUEUR</code>	180	px/s (défini dans <code>Constantes</code>)
<code>MONNAIE_INIT</code>	120	Or de départ (défini dans <code>Constantes</code>)

Algorithmes

Joueur.mettreAJour(deltaMs) :

```

SI directionsActives EST vide ALORS RETOURNER

dx ← 0 ; dy ← 0
SI HAUT DANS directionsActives → dy ← dy - 1
SI BAS DANS directionsActives → dy ← dy + 1
SI GAUCHE DANS directionsActives → dx ← dx - 1
SI DROITE DANS directionsActives → dx ← dx + 1

SI dx = 0 ET dy = 0 ALORS RETOURNER

// Normalisation diagonale (conserve la vitesse à 45°)
len ← sqrt(dx² + dy²)
deplacement ← vitesse × deltaMs / 1000
x ← x + (dx / len) × deplacement
y ← y + (dy / len) × deplacement

```

Le joueur peut donc se déplacer dans 8 directions en combinant des touches (ex. HAUT+DROITE). Le vecteur est normalisé pour que la vitesse diagonale reste égale à la vitesse cardinale.

Camera.centrerSur(joueur) :

```

offsetX ← joueur.x - LARGEUR_VIEWPORT / 2
offsetY ← joueur.y - HAUTEUR_VIEWPORT / 2
offsetX ← CLAMP(offsetX, 0, LARGEUR_CARTE - LARGEUR_VIEWPORT)
offsetY ← CLAMP(offsetY, 0, HAUTEUR_CARTE - HAUTEUR_VIEWPORT)

```

Camera.toScreenX/Y(coordMonde) :

```

screenX ← coordMonde - offsetX
screenY ← coordMonde - offsetY

```

Conditions limites

- Le joueur ne peut pas sortir du monde (clamping dans `carte`).
- La caméra ne peut pas sortir du monde ($\text{offsetX} \in [0, \text{LARGEUR_CARTE} - \text{LARGEUR_VIEWPORT}]$).
- Le déplacement est proportionnel au delta-time pour être indépendant du FPS.

Diagramme de classes (F1)

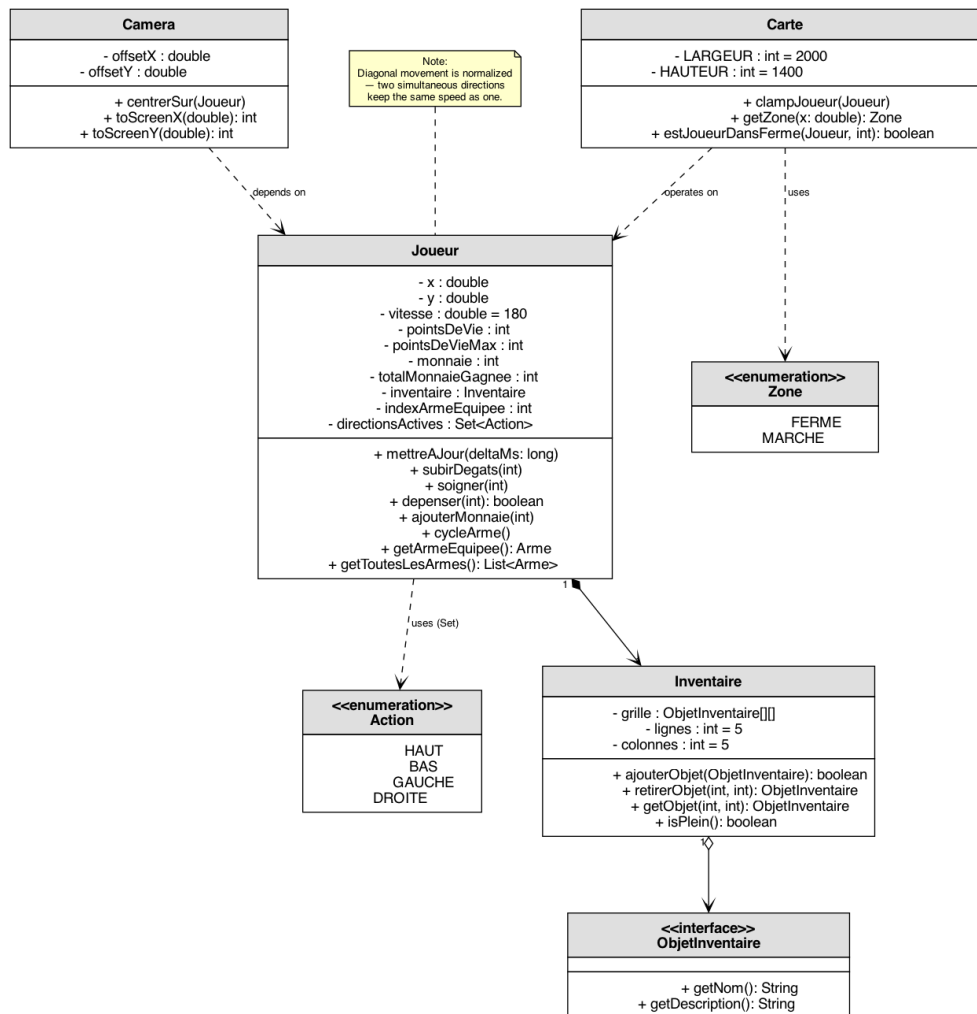


Diagramme de classes F1

5.2 F2 — Monde tilemap et zones

Cette fonctionnalité assure le rendu du monde sous forme d'une grille de tuiles chargées depuis des fichiers texte, divisée en deux zones (ferme à gauche, marché à droite) séparées par un trait animé. Elle est implémentée dans `TileManager` (vue — chargement des 132 images PNG 16 × 16 redimensionnées à 40 × 40 px), `MondeRender` (vue — rendu de la grille avec culling caméra) et `Carte` (modèle — frontières des zones et limites du monde).

Structures de données

TileManager (vue/) : charge 132 images PNG 16×16 px (tiles Kenney Tiny Town), les redimensionne à 40×40 px et les stocke dans un tableau `BufferedImage[]`.

MondeRender (vue/) : lit deux fichiers `.txt` (grilles 25×35 d'entiers), appelle `TileManager` pour récupérer chaque tuile et la dessiner avec culling caméra.

Carte (modele/jeu/) : stocke les dimensions du monde et les limites des zones (ferme : $x \in [0, 1\ 000]$, marché : $x \in [1\ 000, 2\ 000]$).

Constantes du modèle

Constante	Valeur	Description
TAILLE_TUILE	40 px	Taille d’une tuile à l’écran
NB_COLS_ZONE	25	Colonnes par zone
NB_LIGNES	35	Lignes de la carte
LARGEUR_ZONE	1 000	Largeur d’une zone (px)

Algorithme — MondeRenderer.draw(Graphics2D, Camera)

```
POUR ligne DE 0 À NB_LIGNES - 1:
  POUR col DE 0 À NB_COLS_TOTAL - 1:
    worldX ← col × TAILLE_TUILE
    worldY ← ligne × TAILLE_TUILE
    screenX ← camera.toScreenX(worldX)
    screenY ← camera.toScreenY(worldY)

    SI screenX + TAILLE_TUILE < 0 OU screenX > LARGEUR_VIEWPORT ALORS
      CONTINUER ← culling horizontal
    FIN SI

    tileId ← grille[ligne][col]
    image ← tileManager.getTile(tileId)
    g2.drawImage(image, screenX, screenY, TAILLE_TUILE, TAILLE_TUILE, null)
  FIN POUR
FIN POUR

// Trait de séparation animé (x = 1000 en coordonnées monde)
divX ← camera.toScreenX(LARGEUR_CARTE / 2)
Dessiner ligne pointillée bleue à divX
```

Conditions limites

- Le culling évite de dessiner les tuiles hors viewport, optimisant le rendu.
- Si une image de tuile est absente, une couleur de fallback est dessinée.

Diagramme de classes (F2)

5.3 F3 — Ferme et cycle de vie des vaches

Cette fonctionnalité gère le cœur économique du jeu : les vaches achetées au marché sont déployées dans la ferme depuis l’inventaire, puis évoluent automatiquement (Bébé → Adulte → Productive) et génèrent de l’or récolté sans intervention du joueur. Elle est répartie entre `vache` (modèle — machine à états de croissance et accumulation de monnaie), `Ferme` (modèle — collection, tick et récolte du troupeau), `VueFerme` (vue — rendu des sprites et barres de progression) et `ControleurJeu` (contrôleur — appel du tick ferme à chaque frame).

Structures de données

Animal (abstract, modele/ferme/) : classe mère avec `nom` , `x` , `y` , `mettreAJour(deltaMs)` (abstract), `isProductif()` , `getRevenusParCycle()` .

EtatVache (enum) : `BEBE` , `ADULTE` , `PRODUCTIVE` .

vache (extends Animal) :

Attribut	Type	Défaut	Description
<code>etat</code>	<code>EtatVache</code>	<code>BEBE</code>	Phase de croissance
<code>tempsEcoule</code>	<code>long</code>	<code>0</code>	Temps dans la phase courante (ms)
<code>tempsBebeMs</code>	<code>long</code>	<code>10 000</code>	Durée phase Bébé
<code>tempsAdulteMs</code>	<code>long</code>	<code>15 000</code>	Durée phase Adulte
<code>cycleProdMs</code>	<code>long</code>	<code>8 000</code>	Intervalle entre deux productions
<code>revenuParCycle</code>	<code>int</code>	<code>10</code>	Monnaie produite par cycle
<code>monnaieAccumulee</code>	<code>int</code>	<code>0</code>	En attente de récolte

Ferme :

Attribut	Type	Description
<code>vaches</code>	<code>List<Vache></code>	Troupeau
<code>capaciteMax</code>	<code>int</code>	10 max (configurable)

Algorithmes

Vache.mettreAJour(deltaMs) :

```

tempsEcoule ← tempsEcoule + deltaMs
SELON etat:
    BEBE:
        SI tempsEcoule ≥ tempsBebeMs ALORS
            surplus ← tempsEcoule - tempsBebeMs
            etat ← ADULTE ; tempsEcoule ← surplus
        FIN SI
    ADULTE:
        SI tempsEcoule ≥ tempsAdulteMs ALORS
            surplus ← tempsEcoule - tempsAdulteMs
            etat ← PRODUCTIVE ; tempsEcoule ← surplus
        FIN SI
    PRODUCTIVE:
        TANT QUE tempsEcoule ≥ cycleProdMs FAIRE
            monnaieAccumulee ← monnaieAccumulee + revenuParCycle
            tempsEcoule ← tempsEcoule - cycleProdMs
        FIN TANT QUE
    FIN SELON

```

■

Le **report de surplus** garantit qu'un grand delta (lag, pause) ne fait pas perdre de temps de croissance.

Ferme.recolterTout() :

```
total ← 0
POUR CHAQUE vache DANS vaches FAIRE
    total ← total + vache.recolter() // recolter() remet monnaieAccumulee à 0
FIN POUR
RETOURNER total
```

Cette méthode est appelée **automatiquement à chaque frame** par `VuePrincipale.paintComponent()` .

L'or produit est immédiatement crédité au joueur sans aucune action requise de sa part.

Conditions limites

Condition	Comportement
Ferme pleine	<code>ajouterVache()</code> retourne <code>false</code>
Vache null	<code>ajouterVache()</code> retourne <code>false</code>
Récolte sans production	Retourne 0
Delta très grand	Surplus reporté, pas de perte de temps
Alien enlèvement	<code>ferme.enleverDerniereVache()</code> retire la dernière vache

Diagramme de classes (F3)

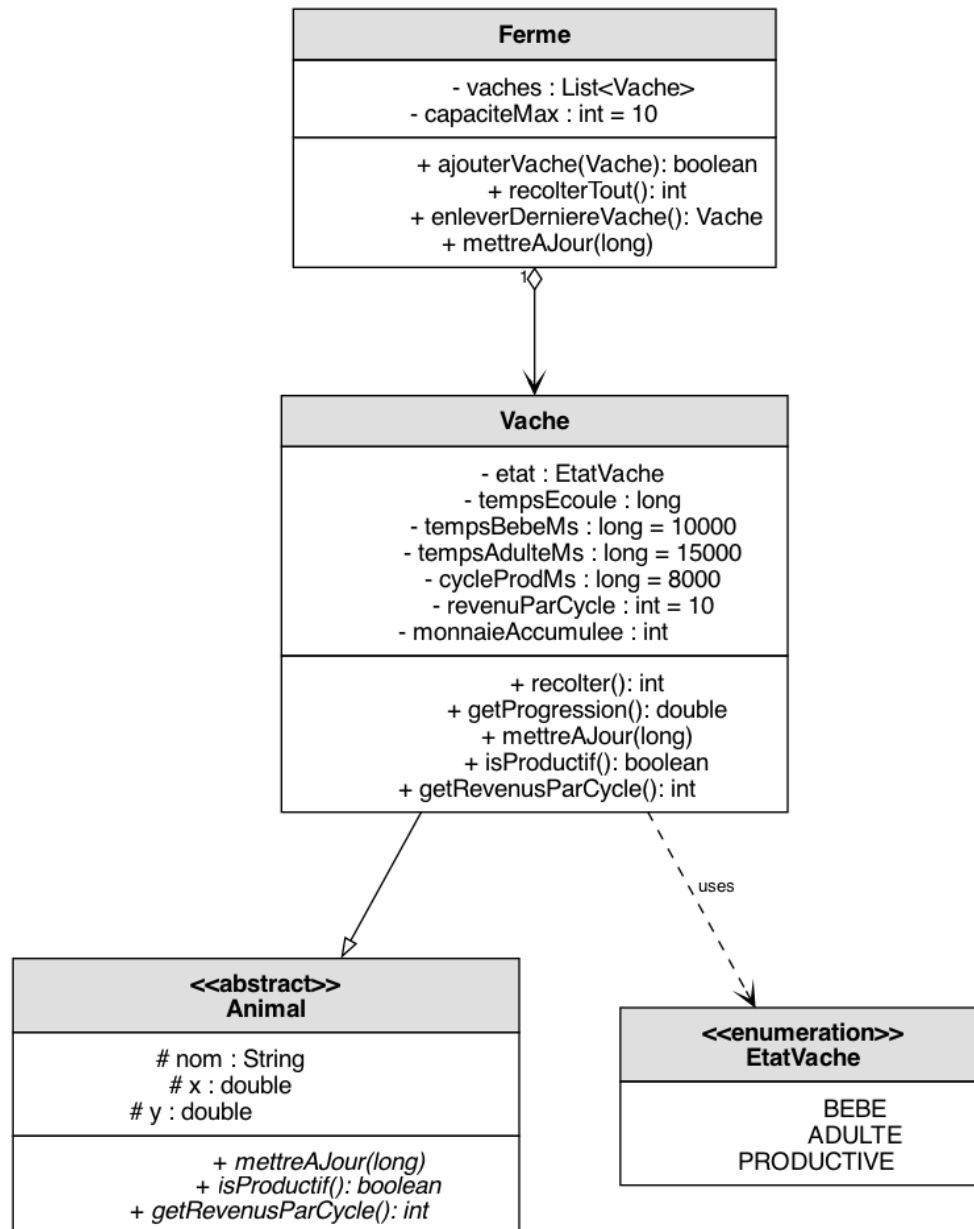


Diagramme de classes F3

5.4 F4 — Système économique : monnaie et marché

Cette fonctionnalité permet au joueur de dépenser son or au marché en s'approchant d'un vendeur, en parcourant sa liste d'articles et en appuyant sur `R` pour acheter — l'objet atterrit alors dans l'inventaire après un court délai d'achat. Elle est implémentée dans `Marche` (modèle — liste des articles disponibles), `ArticleMarche` (modèle — nom, prix, type, niveau requis), `VendeurMarche` (vue — position monde, détection de proximité, sélection d'article), `VueMarchePopup` (vue — anneau pulsé et popup d'achat), `ControleurMarche` (contrôleur — logique d'achat, vérifications, ajout en inventaire) et `ActionDuree` (modèle — barre de progression de l'achat de 800 ms).

Structures de données

Joueur (attributs monnaie) :

Méthode	Comportement
<code>ajouterMonnaie(int)</code>	Ignore les montants ≤ 0
<code>depenser(int)</code>	Retourne false si montant > monnaie ; déduit sinon

ArticleMarche : nom , prix , typeArticle (VACHE / ARME / POTION / BOMBE), niveauRequis .

ResultatAchat (enum): OK , FONDS_INSUFFISANTS , INVENTAIRE_PLEIN , AUCUNE_SELECTION , ARTICLE_VERROUILLE .

VendeurMarche : position monde (x, y), rayon de proximité (90 px), liste d'articles proposés.

ActionDuree : TypeAction.ACHAT (800 ms), progression $0 \rightarrow 1$, barre cyan affichée au-dessus du joueur pendant l'achat.

Articles disponibles au marché

Le marché est une liste plate d'articles accessibles depuis les vendeurs positionnés dans la zone droite. L'**Épée** est donnée gratuitement au démarrage et n'est pas en vente.

Article	Type	Prix	Niveau requis
Vache	VACHE	50g	1
Rayon laser	ARME	120g	1
Potion	POTION	30g	1
Bombe	BOMBE	100g	2
Minigun	ARME	300g	3

Algorithme — *ControleurMarche.acheter(ArticleMarche)*

```

SI article.niveauRequis > partie.getNiveau() ALORS RETOURNER ARTICLE_VERROUILLE
SI joueur.monnaie < article.prix ALORS RETOURNER FONDS_INSUFFISANTS
SI joueur.inventaire.isPlein() ALORS RETOURNER INVENTAIRE_PLEIN

joueur.depenser(article.prix)
controleurJeu.setActionEnCours(new ActionDuree(ACHAT, 800 ms))

SELON article.type:
    VACHE → joueur.inventaire.ajouterObjet(new Vache())
           // La vache est déployée depuis l'inventaire par clic gauche dans la ferme
    ARME → joueur.inventaire.ajouterObjet(arme correspondante)
    POTION → joueur.inventaire.ajouterObjet(new Potion())
    BOMBE → joueur.inventaire.ajouterObjet(new Bombe())
RETOURNER OK

```

Note importante : tous les achats atterrissent dans l'**inventaire** du joueur (grille 5×5). La vache doit ensuite être déployée manuellement par clic gauche quand le joueur est dans la ferme. Les armes doivent être équipées (touche **E** pour cycler).

Conditions limites

- Un achat ne peut démarrer que si l'inventaire n'est pas plein.
- Le joueur doit être à ≤ 90 px du vendeur pour ouvrir son menu.
- Les articles verrouillés (niveau insuffisant) apparaissent grisés dans le popup.

Diagramme de classes (F4)

5.5 F5 — Système de combat : vagues aliens et boss

Cette fonctionnalité orchestre les affrontements contre les extraterrestres : des vagues intermédiaires apparaissent à des moments planifiés sur la barre de progression, et un boss final clôt chaque niveau — le joueur attaque manuellement avec [A] tandis que les aliens ripostent automatiquement selon leurs cooldowns. Elle est répartie entre `Extraterrestre` et `BossFinal` (modèles — stats, types RUNNER/TANK, timer d'abduction, phase 2 boss), `Attaque` (modèle — logique de combat hybride manuel/auto), `ControleurAttaque` (contrôleur — phases APPROCHE/COMBAT/DEPART des vagues), `ControleurCombat` (contrôleur — phases du boss et récompense), `AlienVisuel` (vue-modèle — position et état visuel de chaque alien) et `VueAliens` (vue — rendu des sprites et effets).

Structures de données

Arme : `nom`, `degats` (int), `cooldownMs` (long). Implémente `ObjetInventaire`. Trois constantes statiques :

Constante	Dégâts	Cooldown	DPS
<code>Arme.EPEE</code>	15	1 000 ms	15
<code>Arme.SHOTGUN</code>	50	2 000 ms	25
<code>Arme.MINIGUN</code>	8	250 ms	32

L'Épée est donnée au joueur au démarrage. Les autres s'achètent au marché.

Un champ `indexArmeEquipee` dans `Joueur` pointe vers l'arme active parmi toutes les `Arme` de l'inventaire. `cycleArme()` incrémente cet index ; `getArmeEquipee()` retourne l'arme sélectionnée.

Extraterrestre : `pointsDeVie`, `pointsDeVieMax`, `degats`, `cooldownMs`.

BossFinal (extends `Extraterrestre`) : `recompense`. Fabrique statique `BossFinal.pourNiveau(n)`.

Extraterrestre.TypeAlien (enum) : trois variantes pour les vagues intermédiaires.

Type	PV	Cooldown	Timer abduction	Comportement
NORMAL	base	base	base	Alien standard
RUNNER	×0,6	×0,5	×0,7	Rapide et fragile
TANK	×2,5	×1,5	×1,5	Lent et résistant

Attaque : liste d' Extraterrestre , index alien courant, cooldowns joueur/alien, résultat (`ResultatCombat`), `degatsRecusMulti` (0,45 pour les vagues intermédiaires — réduit les dégâts reçus), `timerAbductionMs` par alien.

AlienVisuel : position monde (x, y), cible, état visuel (`EtatVisuel` : APPROCHE/COMBAT/FUITE/ENLEVEMENT), offset sinusoïdal en combat.

Modèle de combat : hybride manuel/automatique

Le combat est **hybride** : - Le **joueur attaque manuellement** en appuyant sur [A] , ce qui appelle `Attaque.frapperManuel(arme, dommageMulti)` . L'arme a son propre cooldown : si on appuie trop vite, le coup est ignoré.
- Les **aliens attaquent automatiquement** à chaque tick selon leur propre cooldown. - Chaque alien a un **timer d'abduction** : s'il atteint 0 avant d'être tué, il enlève une vache et passe au suivant, même si le joueur a encore des PV.

Algorithme — Attaque.mettreAJour(deltaMs, joueur)

```
SI resultat ≠ EN_COURS ALORS RETOURNER

alien ← aliens[indexAlienCourant]
SI alien = null ALORS resultat ← VICTOIRE ; RETOURNER

// Gestion de l'abduction par timer
alien.timerAbduction ← alien.timerAbduction - deltaMs
SI alien.timerAbduction ≤ 0 ALORS
    vachesAbducteesCeTick++
    indexAlienCourant++
    SI plus d'aliens ALORS resultat ← VICTOIRE
    RETOURNER

// L'alien attaque automatiquement
tempsCooldownAlien ← tempsCooldownAlien - deltaMs
SI tempsCooldownAlien ≤ 0 ET alien vivant ALORS
    degats ← MAX(1, arrondi(alien.degats × degatsRecusMulti))
    joueur.subirDegats(degats)
    tempsCooldownAlien ← alien.cooldownMs
    SI joueur mort ALORS resultat ← DEFAITE
FIN SI
```

Attaque.frapperManuel(arme, dommageMulti) — appelé par [A] :

```
SI resultat ≠ EN_COURS ALORS RETOURNER
SI tempsCooldownJoueur > 0 ALORS RETOURNER ← arme en recharge

degats ← arrondi(arme.degats × dommageMulti)
alien.subirDegats(degats)
totalDegatsInfliges += degats
tempsCooldownJoueur ← arme.cooldownMs

SI alien mort ALORS
    indexAlienCourant++
    SI plus d'aliens ALORS resultat ← VICTOIRE
FIN SI
```

Cycle visuel des phases (ControleurAttaque)

```
INACTIF
  ↓ déclencherVague()
APPROCHE — aliens marchent depuis x=1100 vers la ferme (~250 px/s)
  ↓ alien atteint cible
COMBAT — combat automatique, barre de progression en pause
  ↓ résultat ≠ EN_COURS
DEPART
  ↓ Victoire : aliens fuient à droite (état FUIITE)
  ↓ Défaite : aliens repartent avec une vache (état ENLEVEMENT) → GAME OVER
INACTIF
```

Scaling du boss par niveau — *BossFinal.pourNiveau(n)*

```
pv      ← 80 + n × 40
degats  ← 7 + n × 3
cooldown ← max(600, 1200 - n × 100)
recompense ← 100 + n × 50
```

Phase 2 du boss (Enragé)

Lorsque les PV du boss passent en dessous de 50 % (`getRatioPv()` < 0.5), `ControleurCombat` appelle `boss.enrager()` . À partir de ce moment, `BossFinal.getDegats()` retourne `degats × 1,75` — le boss inflige 75 % de dégâts supplémentaires. Cette mécanique crée un pic de tension en fin de combat.

Mécanique *attaqueSansDefense*

Si le joueur n’est pas dans la zone ferme au moment du déclenchement d’une vague, `attaqueSansDefense = true` . Les aliens arrivent en animation d’approche normale, puis passent directement en état `ENLEVEMENT` et repartent avec une vache, sans qu’aucun combat ne soit possible.

Conditions limites

Condition	Comportement
Dégâts ≤ 0	Ignorés
PV < 0	Plafonnés à 0
Vague vide	Victoire immédiate
Combat terminé	<code>mettreAJour()</code> est sans effet
Bombe sur alien déjà mort	<code>mettreAJour()</code> détecte le PV=0 au tick suivant et avance l’index
Abduction timer écoulé	L’alien enlève une vache et passe au suivant (le joueur garde ses PV)
Joueur hors-ferme au déclenchement	Mode <code>attaqueSansDefense</code> : combat impossible, vache enlevée directement
Défaite vague intermédiaire	Enlève une vache + continue (pas de game over immédiat)
Défaite boss	Game over

Boss < 50 % PV

Phase 2 activée (×1,75 dégâts)

Diagramme de classes (F5)

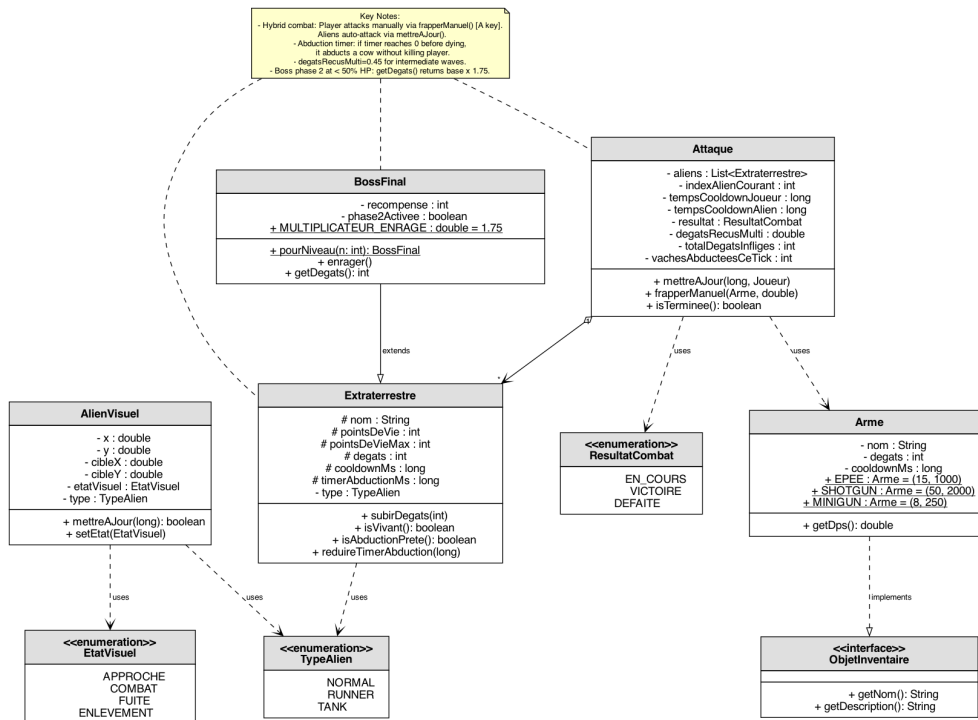


Diagramme de classes F5

5.6 F6 — Progression temporelle et gestion des niveaux

Cette fonctionnalité rythme chaque niveau par une barre de progression temporelle qui déclenche automatiquement les vagues intermédiaires puis le boss final à des instants précis, et fait croître la difficulté d'un niveau à l'autre. Elle est implémentée dans `Niveau` (modèle — fabrique paramétrique de tous les paramètres du niveau : durée, vagues, stats aliens, boss), `EvenementTemporel` (modèle — un événement planifié avec son moment et son type), `BarreProgression` (modèle — chronomètre, liste d'événements, mise en pause lors des combats), `ControleurJeu.tickProgression()` (contrôleur — dispatch des événements et transitions d'état) et `vueBarreProgression` (vue — rendu de la barre avec marqueurs triangulaires).

Structures de données

EvenementTemporel : `momentMs` (long), `type` (`TypeEvenement` : `ATTAQUE_INTERMEDIAIRE` / `COMBAT_FINAL`), `indexVague` (int), `declenche` (boolean).

BarreProgression : liste d' `EvenementTemporel` , `tempsEcoule` , `dureeNiveauMs` , `flag terminee` , `flag enPause` .

Niveau : calcule tous les paramètres du niveau depuis son numéro (durée, nombre de vagues, stats aliens, boss).

Paramètres de scaling

Niveau	Durée	Vagues	Aliens/vague	Alien PV	Boss PV
1	120s	2	1	20	120
2	135s	3	2	30	160
3	150s	4	2	40	200
5	180s	6	3	60	280

Algorithme — *BarreProgression.mettreAJour(deltaMs)*

```

SI enPause ALORS RETOURNER []

tempsEcoule ← tempsEcoule + deltaMs
evenementsDeclenchés ← []

POUR CHAQUE evt DANS evenements FAIRE
    SI evt.declenche = faux ET tempsEcoule ≥ evt.momentMs ALORS
        evt.declenche ← vrai
        evenementsDeclenchés.ajouter(evt)
    FIN SI
FIN POUR

SI tempsEcoule ≥ dureeNiveauMs ALORS terminee ← vrai

RETOURNER evenementsDeclenchés

```

ControleurJeu.tickProgression(deltaMs) :

```

evenements ← barreProgression.mettreAJour(deltaMs)
POUR CHAQUE evt DANS evenements FAIRE
    SI evt.type = ATTAQUE_INTERMEDIAIRE ALORS
        controleurAttaque.declencherVague(evt.indexVague)
        barreProgression.mettreEnPause()
    SI evt.type = COMBAT_FINAL ALORS
        controleurCombat.lancerCombatFinal()
        barreProgression.mettreEnPause()
FIN POUR

SI controleurAttaque.estTermine() ALORS barreProgression.reprendre()
SI controleurCombat.estTermine(VICTOIRE) ALORS passer au niveau suivant
SI controleurCombat.estTermine(DEFAITE) ALORS game over

```

Conditions limites

- La barre est en **pause** pendant tout combat (vague ou boss) — le temps de jeu ne s'écoule pas.
- Un événement ne peut se déclencher qu'une seule fois (`declenche` flag).
- `getProgression()` retourne un ratio $0.0 \rightarrow 1.0$, utilisé par `VueBarreProgression`.

Diagramme de classes (F6)

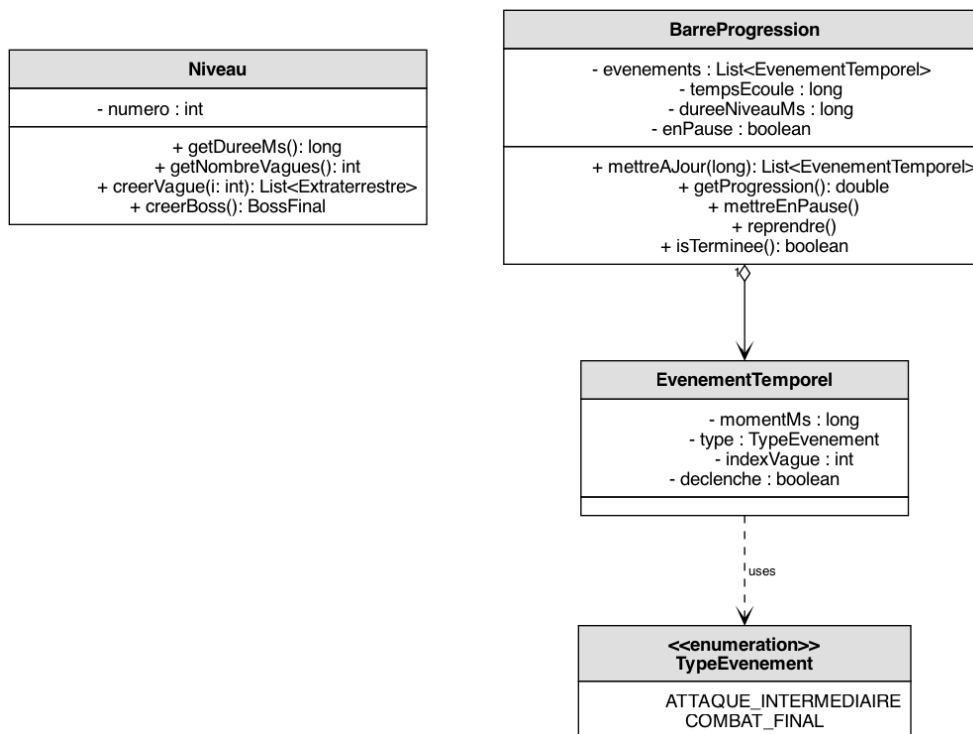


Diagramme de classes F6

5.7 F7 — Boutique d'améliorations

Cette fonctionnalité offre au joueur, après chaque victoire de niveau, la possibilité d'investir son or dans des améliorations permanentes (PV max, dégâts d'arme, vitesse de croissance des vaches, or de départ) avant de passer au niveau suivant. Elle est répartie entre `Upgrades` (modèle — multiplicateurs persistants appliqués à chaque niveau), `EtatJeu.UPGRADE_SHOP` (modèle — état de la machine à états qui suspend le jeu le temps de la boutique), `vueUpgrades` (vue — overlay avec 4 cartes colorées navigables) et `vuePrincipale` (contrôleur-vue — traitement des touches et application des effets d'achat).

Structures de données

Upgrades (modele/joueur/) :

Attribut	Type	Défaut	Description
<code>dommageMulti</code>	double	1.0	Multiplicateur global des dégâts d'armes
<code>cowSpeedMulti</code>	double	1.0	Multiplicateur de vitesse de croissance des vaches
<code>startingGoldBonus</code>	int	0	Or supplémentaire reçu au début de chaque niveau

Les PV max sont appliqués directement sur `Joueur` lors de l'achat (pas de champ dédié dans `Upgrades`).

vueUpgrades (vue/) : 4 cartes colorées, navigation $\leftarrow / \rightarrow$, indicateur triangle sous la carte sélectionnée.

Catalogue des améliorations

Index	Nom	Effet	Coût
0	Max PV +25	<code>joueur.pvMax += 25</code> + soin immédiat	100g
1	Dégâts Arme +10%	<code>upgrades.dommageMulti × 1.10</code>	150g
2	Croissance vache +20%	<code>upgrades.cowSpeedMulti × 1.20</code>	80g
3	Or de départ +50g	<code>upgrades.startingGoldBonus += 50</code>	75g

Flux de la boutique

```

Victoire niveau N
├─
├─ EtatJeu ← UPGRADE_SHOP
│   ├──
│   ├── VuePrincipale.paintComponent() : dessine VueUpgrades en overlay
│   ├── [← →] : naviguer entre les 4 cartes
│   ├── [ENTRÉE] : si joueur.monnaie ≥ cout → achat + application immédiate
│   └─ [ESPACE] : passer → ControleurJeu.initialiserNiveau(N+1)
│       └─ joueur.ajouterMonnaie(startingGoldBonus)

```

Conditions limites

- Si le joueur n'a pas assez d'or, la carte s'affiche en rouge avec « Or insuffisant » et l'achat est bloqué.
- Les multiplicateurs sont **cumulatifs** : deux achats de +10% donnent $\times 1.21$, pas $\times 1.20$.
- L'état `UPGRADE_SHOP` bloque la boucle de jeu normale (pas de tick progression ni de mouvement joueur).

Diagramme de classes (F7)

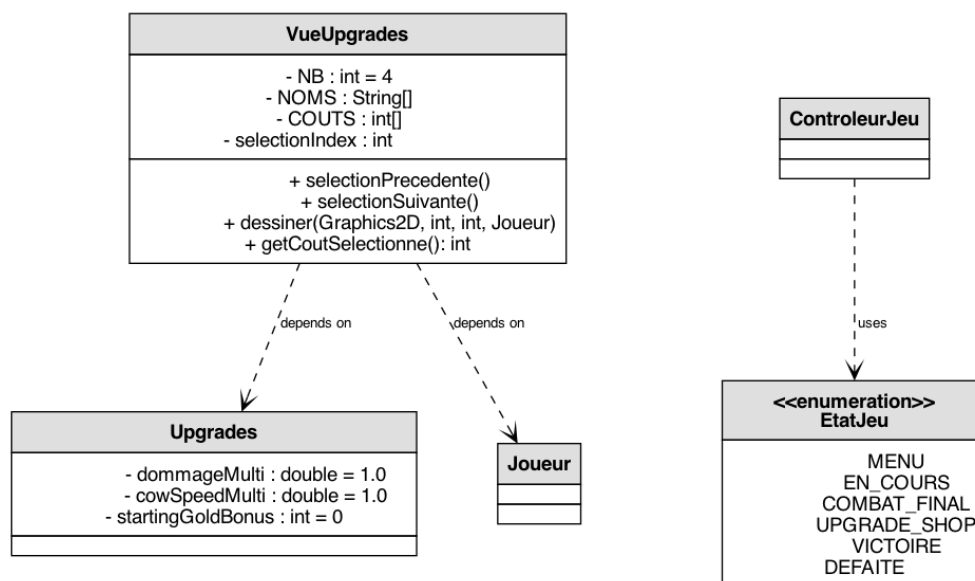


Diagramme de classes F7

5.8 F8 — Tableau des scores

Cette fonctionnalité calcule le score d'un niveau à partir des statistiques de combat (aliens tués, vagues gagnées, vaches perdues, boss vaincu, dégâts infligés), demande les initiales du joueur en fin de partie et persiste le top 10 sur disque. Elle est implémentée dans `scorePartie` (modèle — accumulateur de stats en cours de niveau et formule de score), `TableauScores` (modèle — liste de `TableauScores.Entree` triée, sérialisée dans `~/.alienfarm/scores.dat`), `VueTableauScores` (vue — rendu du classement) et `VuePrincipale.dessinerFinDePartie()` (vue — saisie des initiales via `JOptionPane` et enregistrement).

Structures de données

ScorePartie (modele/jeu/) : traqueur de statistiques **en cours de partie** pour un niveau donné. Accumule : `aliensElimines`, `vaguesGagnees`, `vachesPerdues`, `bossVaincu`, `totalDegatsInfliges`. Calcule le score via `calculerScore()` :

```
score ← aliensElimines × 100
      + vaguesGagnees × 300
      + (bossVaincu ? 2000 × niveau : 0)
      - vachesPerdues × 150
      + totalDegatsInfliges ÷ 5
retourner MAX(0, score)
```

TableauScores : liste de 10 `TableauScores.Entree` max, triée par score décroissant. Sauvegarde/chargement via `ObjectOutputStream` dans `~/.alienfarm/scores.dat`.

TableauScores.Entree (inner class, implémente `Serializable`) : `initiales` (String, 3 chars), `score` (int), `niveau` (int).

Algorithme — `TableauScores.ajouter(initiales, score, niveau)`

```
ini ← initiales.toUpperCase().trim() // tronqué à 3 chars, "???" si vide
nouvEntry ← new Entree(ini, score, niveau)
entrees.ajouter(nouvEntry)
trier entrees par score décroissant
SI entrees.taille() > 10 ALORS supprimer le dernier
sauvegarder()
```

Hooks d'intégration

- `VuePrincipale.dessinerFinDePartie()` : à la fin de partie, `JOptionPane.showInputDialog()` demande les initiales, puis `tableauScores.ajouter(initiales, sc.calculerScore(), partie.getNiveau())`. Appelé via `SwingUtilities.invokeLater()` pour éviter le ré-entrant dans `paintComponent`.
- `VueMenuPrincipal` : bouton « Meilleurs Scores » affiche `vueTableauScores` en overlay depuis le menu d'accueil.

Conditions limites

- Si le fichier `scores.dat` est absent ou corrompu, le tableau démarre vide sans crash.
- Les initiales sont tronquées à 3 caractères, converties en majuscules, remplacées par "???" si vides.

Diagramme de classes (F8)

5.9 F9 — Système de succès

Cette fonctionnalité récompense le joueur lorsqu'il atteint certains seuils de jeu (premier alien tué, 500g gagnés, 50 aliens éliminés, niveau fini à PV max, 3 vaches simultanées) en débloquant des badges affichés dans la barre latérale. Elle est répartie entre `Succes` (modèle — enum de 5 succès avec leur seuil), `GestionnaireSucces` (modèle — compteurs et ensemble des succès débloqués, callback de notification), `vueSucces` (vue — rendu des badges et notifications) et les hooks dans `ControleurJeu`, `ControleurAttaque` et `VuePrincipale.paintComponent()` qui appellent `verifier()` aux bons moments.

Structures de données

Succes (enum, modele/jeu/) :

Succès	Condition	Seuil
PREMIER_SANG	Tuer un premier alien	1
FERMIER_PROSPERE	Gagner 500g au total	500
EXTERMINATEUR	Tuer 50 aliens au total	50
INDESTRUCTIBLE	Finir un niveau à PV maximum	1
RANCHER	Avoir 3 vaches simultanément	3

GestionnaireSucces : Set<Succes> debloquees , Map<Succes, Integer> compteurs .

Algorithme — GestionnaireSucces.verifier(Succes, valeur)

```

SI s DANS debloquees ALORS RETOURNER ← déjà débloqué

compteurs[s] ← compteurs.getOrDefault(s, 0) + valeur
SI compteurs[s] ≥ s.seuil ALORS
    debloquees.ajouter(s)
    afficherNotification(s)
FIN SI

```

Hooks d'intégration

- ControleurAttaque : sur alien tué → `verifier(PREMIER_SANG, 1) + verifier(EXTERMINATEUR, 1)` .
- ControleurJeu.tickProgression : fin de niveau + joueur PV max → `verifier(INDESTRUCTIBLE, 1)` .
- Joueur.ajouterMonnaie(montant) : → `verifier(FERMIER_PROSPERE, montant)` .
- VuePrincipale.paintComponent : si `ferme.getNombreAnimaux() ≥ 3` → `verifier(RANCHER, 3)` .

Conditions limites

- Un succès déjà débloqué ne peut pas être déclenché une seconde fois.
- Les succès sont sauvegardés avec le tableau des scores (même fichier de persistance).

Diagramme de classes (F9)

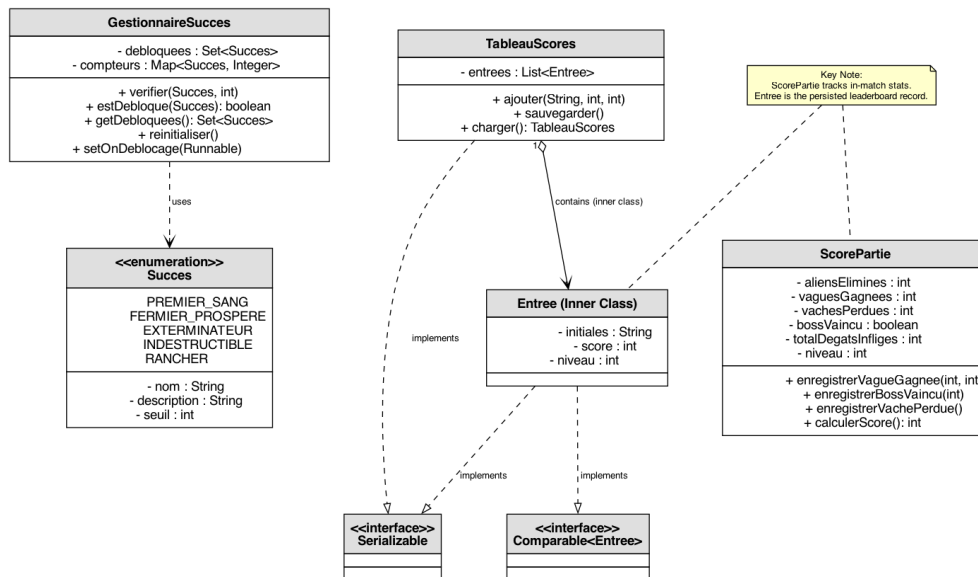


Diagramme de classes F9

Diagramme de classes global

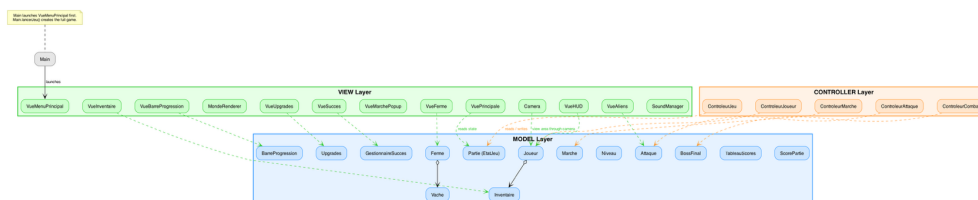


Diagramme de classes global

6. Résultats

Le projet aboutit à un jeu complet et jouable, intégrant toutes les fonctionnalités prévues.

Menu principal et tableau des scores

L'écran d'accueil présente le titre, un bouton « Jouer » et un bouton « Meilleurs Scores » qui affiche le tableau des 10 meilleures parties (initiales, score, niveau atteint).

Ferme et déplacement

Le joueur se déplace librement sur le monde tilemap. Les vaches apparaissent dans la zone ferme avec des sprites distincts selon leur stade (Bébé marron, Adulte noir et blanc, Productive dorée). Des étiquettes d'état et des barres de progression sont affichées au-dessus de chaque animal. La caméra suit le joueur avec un effet fluide.

Marché et achats

En zone marché, les 4 vendeurs sont des objets positionnés dans le monde. Quand le joueur s'approche à moins de 90 px, un anneau pulsé jaune s'affiche avec le hint « [R] acheter ». Le popup d'achat (en bas au centre) liste les articles, leurs prix et les verrous de niveau. Pendant un achat, une barre cyan apparaît au-dessus du joueur.

Combat — Vague alien

Lors d'une vague, les aliens (sprites Kenney UFO manned) approchent depuis le bord droit, tremblent en phase COMBAT, puis fuient (victoire) ou repartent avec une vache (défaite). L'overlay semi-transparent affiche les barres de PV des deux camps.

Combat final — Boss

Le boss (alien plus grand, rose, avec variantes de dégâts visuels) approche en fin de barre de progression. Le résultat — VICTOIRE ou DÉFAITE — s'affiche en overlay avec le gain d'or associé.

Boutique d'améliorations

Entre les niveaux, l'overlay de la boutique présente 4 cartes colorées. La carte sélectionnée est surlignée avec sa couleur thématique (bleu HP, rouge dégâts, vert vaches, or monnaie). Le coût s'affiche en rouge si les fonds sont insuffisants.

Tests unitaires

179 tests JUnit 5 passent sans erreur :

Classe de test	Tests
JoueurTest	25
PartieTest	23
CarteTest	18
MarcheTest	9
FermeTest	14
VacheTest	12
ActionDureeTest	10
NiveauTest	11

BarreProgressionTest	12
CombatUnitTest	11
AttaqueTest	9
ControleurAttaqueTest	9
ControleurCombatTest	6
ControleurMarcheTest	10
Total	179

[INFO] Tests run: 179, Failures: 0, Errors: 0, Skipped: 0

[INFO] BUILD SUCCESS

Correctifs et améliorations récents (avril 2026)

Les améliorations suivantes ont été intégrées après la version initiale du rapport :

- **Fenêtre et UI adaptatives** : la fenêtre est ajustée automatiquement à l'écran disponible ; la barre de progression est ancrée à la hauteur réelle du panneau pour rester visible.
- **Audio centralisé** : ajout d'un `SoundManager` (thèmes menu/exploration/combat, clics, jingle) avec chargement robuste et fallback si un fichier est indisponible.
- **Musique de combat fiabilisée** : le thème combat est réappliqué tant qu'une vague ou le boss est actif.
- **Combat intermédiaire rééquilibré** : réduction des dégâts reçus, plafonnement du nombre d'aliens simultanés et temps d'abduction plus permissif en début de jeu.
- **Bug critique boss corrigé** : si un alien/boss meurt via un effet externe (ex. bombe), le combat se finalise correctement (plus de blocage à PV affichés à 0).
- **Détection ferme harmonisée** : ajout d'une tolérance de frontière (marge) pour éviter les faux messages « hors ferme » près de la séparation ferme/marché.
- **Déploiement des vaches** : placement en grille compacte au centre de la ferme (au lieu d'un placement aléatoire dispersé).
- **Inventaire enrichi** : affichage des sprites des objets (vache, potion, bombe, rayon laser) à la place des lettres quand les assets existent.
- **Ergonomie des contrôles** :
 - suppression de la pause clavier et de son hint ;
 - ajout de raccourcis rapides inventaire : `w` (potion), `x` (bombe) ;
 - clarification des consignes d'inventaire (clic gauche sur l'objet).
- **Attaque manuelle plus cohérente** : pendant une vague active et défendable, l'appui sur `A` est pris en compte sans faux « alien trop loin ».

7. Documentation utilisateur

Prérequis

1. **Java 17+** : Vérifier avec `java --version` .
2. **Apache Maven 3.8+** : Vérifier avec `mvn --version` .

Compilation et lancement

Dans le dossier `farmdefense/` :

```
mvn compile exec:java
```

Ou en deux étapes :

```
mvn compile
java -cp target/classes com.fermedefense.Main
```

Comment jouer

Objectif

Gérer votre ferme, acheter des armes et défendre vos vaches contre les extraterrestres. Survivez le plus longtemps possible et battez le boss de fin de niveau pour progresser.

Déplacement

Touche	Action
Flèches directionnelles ou Z/Q/S/D	Déplacer le fermier
A	Attaquer avec l'arme équipée
E	Changer d'arme équipée
W	Utiliser rapidement une potion (si disponible)
X	Utiliser rapidement une bombe (si disponible)

Le monde est divisé en deux zones : - **Gauche (0–1 000 px)** : la ferme — vos vaches s'y trouvent. - **Droite (1 000–2 000 px)** : le marché — achetez vaches et armes.

Gestion de la ferme

- Vos vaches évoluent automatiquement : **Bébé** (marron) → **Adulte** (noir et blanc) → **Productive** (dorée).
- En phase **Productive**, elles accumulent de l'or visible comme un badge au-dessus d'elles.
- La récolte est **automatique** (plus besoin d'appuyer sur une touche).

- Pour déployer une vache depuis l'inventaire : cliquez sur sa case en étant dans la ferme.

Achats au marché

- Approchez-vous d'un vendeur (un anneau jaune pulsé apparaît).
- Appuyez sur **R** pour ouvrir le menu d'achat.
- Naviguez avec **HAUT/BAS**, achetez avec **R** (un achat prend 1,5 secondes).
- Les articles grisés nécessitent un niveau supérieur au vôtre.

Combat

- Des **vagues d'aliens** apparaissent automatiquement selon la barre de progression en bas de l'écran (marqueurs triangulaires jaunes).
- Soyez dans la ferme pour défendre vos vaches.
- Pendant les vagues et le boss, utilisez **A** pour frapper avec l'arme équipée.
- Si vous perdez un combat intermédiaire, un alien repart avec une vache. Si vous perdez le boss final, c'est **Game Over**.

Inventaire (rappel rapide)

- **Clic gauche** sur un objet de l'inventaire :
 - Vache → déploiement à la ferme
 - Arme → utilisation/attaque
 - Potion → soin
 - Bombe → attaque instantanée
- Raccourcis rapides : **w** (potion), **x** (bombe).

Boutique d'améliorations

- Après chaque victoire de niveau, une boutique apparaît.
- Naviguez avec **← →**, achetez avec **ENTRÉE**, passez au niveau suivant avec **ESPACE**.

Fin de partie

- En cas de Game Over, saisissez vos **3 initiales** pour enregistrer votre score.
- Consultez le tableau des meilleurs scores depuis le menu principal.

8. Documentation développeur

Cette section s'adresse à toute personne souhaitant reprendre, améliorer ou étendre le projet.

8.1 Point d'entrée et classes à explorer en premier

Classe	Paquet	Rôle
Main	com.fermedefense	Point d'entrée. Lance VueMenuPrincipal ; Main.lancerJeu() instancie le modèle et la vue. À lire en premier.
VueMenuPrincipal	vue/	Écran d'accueil animé (vaches tournantes, PressStart2P) avec boutons Jouer / Meilleurs Scores.
VuePrincipale	vue/	JFrame principal + boucle Timer 60 FPS + KeyListener + PanneauJeu (inner class).
ControleurJeu	controleur/	Orchestrateur : gère le tick global, la progression et les transitions d'état.
Partie	modele/jeu/	Machine à états (EtatJeu) — point de vérité de l'état du jeu.
Constantes	utilitaire/	Toutes les constantes globales modifiables.
SoundManager	utilitaire/	Gestionnaire audio centralisé (thèmes menu/exploration/combat, effets).

8.2 Arborescence du projet

```

farmdefense/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/com/fermedefense/
│   │   │   ├── Main.java
│   │   │   └── modele/
│   │   │       ├── jeu/           ← Partie, EtatJeu, Carte, Zone,
│   │   │       │               TableauScores, Succes, GestionnaireSucces
│   │   │       ├── joueur/       ← Joueur, Action, ActionDuree, Upgrades
│   │   │       ├── ferme/        ← Animal, Vache, EtatVache, Ferme
│   │   │       ├── marche/       ← Marche, ArticleMarche, TypeArticle, ResultatAchat
│   │   │       ├── combat/       ← Arme, Extraterrestre, BossFinal, Attaque,
│   │   │       │               AlienVisuel, EtatVisuel, ResultatCombat
│   │   │       └── progression/ ← Niveau, BarreProgression, EvenementTemporel
│   │   └── vue/
│   │       ├── VuePrincipale.java ← Fenêtre + boucle + input
│   │       ├── Camera.java        ← Monde → écran
│   │       ├── MondeRenderer.java ← Tilemap avec culling
│   │       ├── TileManager.java   ← Chargement PNG tuiles
│   │       └── VueFerme.java       ← Rendu vaches + sprites

```

```

├── VueHUD.java          ← Barre HP, or, niveau, timer
├── VueMarche.java       ← Objets vendeurs monde
├── VendeurMarche.java   ← Un vendeur (position, items)
├── VueMarchePopup.java  ← Anneau + popup achat
├── VueAliens.java       ← Dessine aliens/boss
├── CacheSpritesAliens.java ← Chargement PNGs aliens
├── VueIndicateurs.java  ← Flèches de navigation bord
├── VueCombat.java       ← Overlay HP combat
├── VueBarreProgression.java ← Timer + marqueurs
├── VueActionJoueur.java ← Barre cooldown au-dessus joueur
├── VueUpgrades.java     ← Boutique overlay
├── VueTableauScores.java ← Rendu top 10
├── VueEffetHit.java     ← Burst d'impact
├── VueEffetTexte.java   ← Textes flottants (+Xg, -X)
├── VueInventaire.java   ← Grille inventaire sidebar
├── controleur/
│   ├── ControleurJeu.java ← Boucle principale + niveau
│   ├── ControleurJoueur.java ← Mouvement + récolte
│   ├── ControleurAttaque.java ← Vagues + phases visuelles
│   ├── ControleurCombat.java ← Boss + phases visuelles
│   └── ControleurMarche.java ← Logique d'achat
├── utilitaire/
│   └── Constantes.java
├── resources/
│   ├── images/
│   │   ├── tiles/tt/          ← tile_0000.png ... tile_0131.png
│   │   ├── player/           ← boy_{down,up,left,right}_{1,2}.png
│   │   └── aliens/            ← shipGreen_manned.png, etc.
│   └── maps/
│       ├── farm_map.txt       ← Grille 25×35 zone ferme
│       └── market_map.txt     ← Grille 25×35 zone marché
├── test/java/com/fermedefense/ ← 179 tests JUnit 5
├── docs/                      ← Documentation + images rapport
├── HANDOFF.md
├── PLAN.md
└── RAPPORT.md                 ← Ce fichier

```

8.3 Constantes principales à modifier

Toutes les constantes se trouvent dans `utilitaire/Constantes.java` ou comme champs statiques des classes concernées.

Constante	Classe	Effet
LARGEUR_VIEWPORT / HAUTEUR_VIEWPORT	Constantes	Taille de la fenêtre de jeu
PLAYER_SIZE	VuePrincipale	Taille du sprite joueur
vitesse (Joueur)	Joueur	Vitesse de déplacement (px/s)
TAILLE_TUILE	Constantes	Taille d'une tuile (40 px par défaut)
tempsBebems / tempsAdultems	Vache	Durée des phases de croissance
cycleProdMs / revenuParCycle	Vache	Cadence et montant de production

RAYON	VendeurMarche	Rayon de proximité vendeur (90 px)
Arme.EPEE / SHOTGUN / MINIGUN	Arme	Stats des trois armes (dégâts + cooldown)
BossFinal.pourNiveau(n)	BossFinal	Formule de scaling du boss
COUTS[]	VueUpgrades	Prix des 4 améliorations
MONNAIE_INIT	Constantes	Or de départ (120g par défaut)

8.4 Ajouter un nouvel upgrade

1. Ajouter les constantes (nom, coût, couleur, description) dans `VueUpgrades` en incrémentant `NB`.
2. Si cela nécessite un multiplicateur persistant, l'ajouter dans `Upgrades.java`.
3. Appliquer l'effet dans `vuePrincipale.actionKeyListener` (case `UPGRADE_SHOP`, touche `ENTRÉE`).
4. Si le bonus doit s'appliquer chaque niveau, l'intégrer dans `ControleurJeu.initialiserNiveau()`.

8.5 Ajouter un nouveau succès

1. Ajouter l'entrée dans l'enum `Succes` avec son nom, description et seuil.
2. Appeler `gestionnaireSucces.verifier(Succes.MON_SUCCES, valeur)` à l'endroit approprié dans le code.
3. Le reste (déclenchement, notification, sauvegarde) est géré automatiquement par `GestionnaireSucces`.

8.6 Fonctionnalités non implémentées — perspectives

1. **Nouvelles armes et ennemis** — Le modèle `Arme` est générique (dégâts + cooldown). Il suffit d'ajouter des constantes statiques supplémentaires, de les enregistrer dans `Marche`, et de les référencer dans `ControleurMarche.acheter()`. Pour de nouveaux types d'aliens, étendre `TypeAlien` dans `Extraterrestre` et ajuster `Niveau.creerVagueDynamique()`.
2. **Animations de transitions** — Fondu (alpha progressif) lors des changements d'état (victoire → boutique → niveau suivant). Implémenter dans `vuePrincipale.paintComponent()` avec un timer de fade piloté par `EtatJeu`.
3. **Mode survie infini** — Remplacer la condition de fin de niveau (`barreProgression.isTerminee()`) par une difficulté croissante sans limite : les vagues s'enchaînent indéfiniment et le boss réapparaît périodiquement.
4. **Mode multijoueur local** — L'architecture MVC sépare clairement le modèle de la vue ; un second `ControleurJoueur` lisant des touches différentes pourrait contrôler un second `Joueur` dans le même monde.

9. Conclusion et perspectives

Réalisations

Ce projet a abouti à un jeu de stratégie/défense complet et jouable. Toutes les fonctionnalités prévues au cahier des charges ont été implémentées : déplacement libre dans un monde tilemap de $2\,000 \times 1\,400$ px avec caméra, cycle de vie des vaches (Bébé → Adulte → Productive), système économique avec 4 vendeurs en monde ouvert, vagues d'aliens avec phases visuelles animées (approche/combat/départ), boss final avec scaling par niveau, boutique d'améliorations entre les niveaux, tableau des meilleurs scores persistant sur disque, et 5 succès débloquables. L'ensemble est couvert par **179 tests unitaires** JUnit 5 qui passent sans erreur.

Difficultés rencontrées et solutions

La principale difficulté a été la **gestion de la boucle de jeu temps réel** avec le modèle événementiel de Swing. L'utilisation d'un `Timer` Swing (plutôt qu'un thread séparé) garantit que tous les accès au modèle se font sur l'EDT, évitant les problèmes de concurrence sans recourir à des `synchronized` complexes.

Une seconde difficulté a été la **coordination des phases visuelles** des combats (approche/combat/départ) avec la logique de jeu. Cela a été résolu par une machine à états `PhaseAttaque` dédiée dans `ControleurAttaque`, indépendante de la logique de combat (`Attaque`), ce qui permet de tester les deux séparément.

La **gestion de la caméra** dans un monde plus grand que le viewport a nécessité une réflexion soignée sur la séparation coordonnées-monde / coordonnées-écran. La classe `camera` centralisée, utilisée par tous les renderers, a évité les bugs de décalage.

Apprentissages

- Maîtrise de l'architecture **MVC** dans un jeu temps réel avec Swing.
- Conception d'une **machine à états** pour orchestrer des flux complexes (combats en phases, transitions entre états de jeu).
- Utilisation avancée de `Graphics2D` (sprites, `RenderingHints`, transformations de coordonnées, overlays semi-transparents).
- Importance du **delta-time** pour un mouvement indépendant du FPS.
- Valeur des **tests unitaires** : les 179 tests ont permis de détecter des régressions à plusieurs reprises, notamment sur la logique de combat et la barre de progression.

Perspectives

L'architecture du projet est extensible. Les évolutions naturelles seraient :

- **Nouvelles armes et ennemis** : le modèle `Arme` et `Extraterrestre` sont génériques et facilement étendables.
- **Mode survie infini** : remplacer la condition de fin de niveau par une difficulté croissante sans limite.
- **Animations de transitions** : fondu lors des changements d'état (victoire → boutique → niveau suivant).
- **Mode multijoueur local** : second `ControleurJoueur` sur des touches différentes pour un deuxième fermier.